1 Dokumentacja funkcjonalna - Aplikacja w języku C

1.1 Cel projektu

Głównym celem projektu jest stworzenie konsolowej aplikacji w języku C, która potrafi „ładnie" i czytelnie rozrysować graf na płaszczyźnie. Program przyjmuje surowe dane o wierzchołkach oraz łączących je krawędziach, a następnie oblicza dla nich takie współrzędne, aby wygenerowany obraz był przejrzysty i miał jak najmniej przecinających się linii.

W ramach tej części projektu skupiamy się na trzech głównych zadaniach:

- **Przetwarzaniu danych:** Poprawnym wczytaniu pliku tekstowego z opisem grafu i wygenerowaniu gotowych współrzędnych w formacie tekstowym lub binarnym (aby przygotować grunt pod drugą część projektu w Javie).
- **Algorytmice:** Zaimplementowaniu dwóch różnych matematycznych metod rozmieszczania węzłów (algorytmu Fruchtermana-Reingolda oraz algorytmu Tutte'a), co pozwoli nam w praktyce porównać ich skuteczność.
- **Dobrych praktykach inżynierskich:** Bieżącym wersjonowaniu kodu przy użyciu systemu Git oraz tworzeniu przejrzystej dokumentacji w środowisku LaTeX.

1.2 Opis funkcji i algorytmów

Aplikacja w języku C działa wyłącznie w trybie tekstowym (linia poleceń). Implementuje dwa algorytmy rozmieszczania:

1.2.1 Algorytm Fruchterman-Reingold

Algorytm Fruchterman-Reingold jest klasycznym podejściem z grupy metod siłowych (*force-directed layouts*). Traktuje on graf jako układ fizyczny, w którym wierzchołki podlegają prawom mechaniki klasycznej.

Model fizyczny W modelu tym przyjmuje się dwie podstawowe interakcje między elementami grafu:

- **Siła odpychania (f_r):** Działa między każdą parą wierzchołków w grafie, zapobiegając ich nakładaniu się. Jest wzorowana na prawie Coulomba dla ładunków jednoimiennych.

- **Siła przyciągania (f_a):** Działa wyłącznie między wierzchołkami połączonymi krawędzią, dążąc do utrzymania ich w bliskiej odległości. Jest wzorowana na prawie Hooke'a (model sprężyny).

Podstawy matematyczne Kluczowym elementem algorytmu jest wyznaczenie optymalnej odległości między wierzchołkami, oznaczonej jako k, która zależy od powierzchni obszaru rysowania ($W \times H$) oraz liczby wierzchołków ($|V|$):

$$k = \sqrt{\frac{W \cdot H}{|V|}}$$

Dla wierzchołków oddalonych o dystans d, wartości sił obliczane są według wzorów:

$$f_a(d) = \frac{d^2}{k}, \quad f_r(d) = \frac{k^2}{d}$$

Proces iteracyjny i chłodzenie Algorytm działa w pętli, aktualizując pozycje wierzchołków o wektor wypadkowy sił. Aby układ ustabilizował się i przestał drgać w końcowej fazie, wprowadzany jest parametr temperatury t (*cooling schedule*).

W każdej iteracji maksymalne przesunięcie wierzchołka jest ograniczane przez aktualną temperaturę:

$$v_{pos} = v_{pos} + \frac{\vec{F}}{|\vec{F}|} \cdot \min(|\vec{F}|, t)$$

Temperatura maleje liniowo lub wykładniczo w każdej iteracji, co pozwala na duże zmiany układu na początku obliczeń i precyzyjne dociągnięcie wierzchołków do stanu równowagi na końcu.

LINK DO WIZUALIZACJI

1.2.2 Twierdzenie Tutte'a o sprężynach

Twierdzenie Tutte'a (1963) stanowi fundament teoretyczny dla wizualizacji grafów planarnych. Gwarantuje ono, że dla każdego grafu planarnego 3-spójnego istnieje osadzenie na płaszczyźnie, w którym wszystkie krawędzie są odcinkami prostymi, a ściany wewnętrzne są wielokątami wypukłymi.

Metoda wag barycentrycznych Istota algorytmu polega na potraktowaniu krawędzi grafu jako idealnych sprężyn o zerowej długości spoczynkowej. Proces wyznaczania współrzędnych przebiega w dwóch etapach:

1. **Ustalenie otoczki:** Wybierany jest cykl zewnętrzny grafu (ściana zewnętrzna), a jego wierzchołki są na sztywno przypisywane do pozycji tworzących wielokąt wypukły na płaszczyźnie.

2. **Wyznaczenie wnętrza:** Pozycje wszystkich wierzchołków wewnętrznych są wyznaczane jako średnia arytmetyczna pozycji ich sąsiadów.

Sformułowanie matematyczne Dla każdego wierzchołka wewnętrznego v_i, jego współrzędne (x_i, y_i) muszą spełniać następujący warunek równowagi:

$$x_i = \frac{1}{\deg(v_i)} \sum_{j \in Adj(v_i)} x_j, \quad y_i = \frac{1}{\deg(v_i)} \sum_{j \in Adj(v_i)} y_j$$

Gdzie $\deg(v_i)$ to stopień wierzchołka, a $Adj(v_i)$ to zbiór jego sąsiadów.

Układ równań liniowych Problem wyznaczenia współrzędnych sprowadza się do rozwiązania układu równań liniowych, który można zapisać macierzowo przy użyciu macierzy Laplaciana L:

$$L\mathbf{x} = \mathbf{b}_x, \quad L\mathbf{y} = \mathbf{b}_y$$

Wektor $\mathbf{b}$ zawiera informacje o ustalonych pozycjach wierzchołków zewnętrznej otoczki. Ponieważ macierz układu jest rzadka i symetryczna, rozwiązanie można uzyskać efektywnie przy użyciu metod takich jak eliminacja Gaussa lub metoda sprzężonych gradientów.

LINK DO WIZUALIZACJI

1.3 Instrukcja obsługi – Jak uruchomić program?

Aplikacja nie posiada okienek ani przycisków – sterujemy nią za pomocą wpisywania komend w terminalu (konsoli). Jest to szybki sposób, który pozwala komputerowi skupić się wyłącznie na obliczeniach.

1.3.1 Krok 1: Przygotowanie pliku z grafem

Zanim uruchomisz program, musisz mieć plik tekstowy (np. `moj_graf.txt`), w którym opiszesz połączenia. Każda linia w pliku to jedna „krawędź" (połączenie między dwoma punktami). Format to:
`Nazwa_Połączenia Punkt_A Punkt_B Waga`

Przykład zawartości pliku:

```
Krawedz1 1 2 1.5
Krawedz2 2 3 1.0
```

1.3.2 Krok 2: Uruchomienie programu

Otwórz terminal w folderze z programem i wpisz komendę według poniższego wzoru:

```
./program_c -i moj_graf.txt -o wyniki.txt -a 0
```

1.3.3 Krok 3: Zrozumienie opcji (flag)

Program potrzebuje od Ciebie kilku informacji, które przekazujesz za pomocą liter (tzw. flag):

- `-i` (Input) – Po tej literze wpisz nazwę swojego pliku z danymi.
- `-o` (Output) – Tutaj wpisz, jak ma się nazywać plik, który program stworzy (tam będą gotowe współrzędne).
- `-a` (Algorytm) – Wybierz, jak program ma liczyć:
  - `0` – tryb „siłowy" (rozpychanie kropek, aby było ładnie),
  - `1` – tryb „matematyczny" (układanie wewnątrz figur).
- `-b` (Binary) – Jeśli dodasz tę literę na końcu, program zapisze wynik w formacie „tylko dla komputera" (binarnym). Jeśli jej nie dasz, plik będzie zwykłym tekstem, który możesz otworzyć w Notatniku.

1.4 Format danych wejściowych i wyjściowych

Aby dane były łatwe do odczytania zarówno dla komputera, jak i dla człowieka, program korzysta z prostego formatu tekstowego. Poszczególne informacje w każdej linii są oddzielone od siebie znakami białymi (spajami).

1.4.1 Format wejściowy (Lista krawędzi)

Plik, który przygotowujesz dla programu, musi zawierać opis połączeń w grafie. Każda linia pliku odpowiada jednej krawędzi.

Format zapisu w pojedynczej linii:

- `<nazwa_krawędzi> <id_punktu_A> <id_punktu_B> <waga_połączenia>`

Przykład pliku wejściowego:

```
AB 1 2 1.0
BC 2 3 1.0
CD 3 4 1.4
```

1.4.2 Format wyjściowy (Współrzędne punktów)

Po wykonaniu obliczeń program domyślnie stworzy plik tekstowy z gotowymi pozycjami dla każdego wierzchołka. Taki plik można łatwo otworzyć i zweryfikować w dowolnym edytorze tekstu. Wartości w poszczególnych liniach oddzielone są spacjami.

Format zapisu w pojedynczej linii:

- `<id_wierzchołka> <współrzędna_x> <współrzędna_y>`

Przykład pliku wyjściowego:

```
1 0.0 0.5
2 1.2 0.0
3 1.2 1.5
```

1.4.3 Opcjonalny format binarny

Jeśli zależy nam na wydajności przy przetwarzaniu bardzo dużych grafów, program może zapisać wyniki w zoptymalizowanym formacie binarnym. Taki plik nie jest bezpośrednio czytelny dla człowieka (zawiera surowe zrzuty danych z pamięci), ale jego zastosowanie jest kluczowe przy przekazywaniu wyników do innych aplikacji lub kolejnych modułów analitycznych. Pozwala to wyeliminować czasochłonne parsowanie tekstu i gwarantuje błyskawiczne wczytywanie danych.

1.5 Sytuacje wyjątkowe i kody błędów

Program został zabezpieczony przed podstawowymi błędami ze strony użytkownika oraz problemami systemowymi. W przypadku wystąpienia błędu, program przerywa działanie, wyświetla odpowiedni komunikat na standardowym wyjściu błędów (`stderr`) i zwraca kod błędu inny niż `0`.

- **Kod 1 - Brak wymaganych argumentów:**
  `Błąd: Nie podano pliku wejściowego lub wyjściowego.`
  Przyczyna: Użytkownik nie podał wymaganych flag `-i` lub `-o` przy uruchamianiu programu.

- **Kod 2 - Błąd odczytu pliku:**
  `Błąd: Nie można otworzyć pliku wejściowego.`
  Przyczyna: Wskazany plik z grafem nie istnieje, ścieżka jest nieprawidłowa lub program nie ma uprawnień do odczytu.

- **Kod 3 - Niepoprawny format danych:**
  `Błąd: Niepoprawny format pliku wejściowego.`
  Przyczyna: Plik z grafem zawiera nieobsługiwane znaki, brakuje parametrów lub wagi krawędzi nie są wartościami liczbowymi.

- **Kod 4 - Niewłaściwe parametry flag:**
  `Błąd: Niewłaściwa wartość argumentu.`
  Przyczyna: Podano nieznany numer algorytmu (inny niż 0 lub 1) lub dodano nieznaną flagę.

- **Kod 5 - Błąd alokacji pamięci:**
  `Błąd krytyczny: Brak pamięci do zaalokowania grafu.`
  Przyczyna: Program nie był w stanie przydzielić wymaganej pamięci dla bardzo dużego grafu.

2 Dokumentacja implementacyjna – Architektura i logika systemu

Niniejszy rozdział zawiera szczegółowy opis techniczny implementacji aplikacji w języku C. Dokumentacja ta ma na celu ułatwienie zrozumienia struktury kodu, sposobu zarządzania pamięcią oraz logiki przepływu danych pomiędzy poszczególnymi modułami systemu.

2.1 Modułowość i struktura plików

Projekt został zaprojektowany zgodnie z zasadą dekompozycji funkcjonalnej. Kod źródłowy jest podzielony na logiczne jednostki (pliki nagłówkowe `.h` i pliki implementacji `.c`), co pozwala na niezależne rozwijanie warstwy obliczeniowej, komunikacyjnej oraz zarządzania danymi:

- **Moduł sterujący (`main.c`):** Centralny punkt aplikacji odpowiedzialny za interakcję z systemem operacyjnym, parsowanie argumentów oraz orkiestrację procesów.

- **Moduł struktur danych (`graph.h`):** Definiuje abstrakcyjne typy danych reprezentujące graf (wierzchołki i krawędzie).

- **Moduł wejścia/wyjścia (`io.c`, `io.h`):** Odpowiada za niskopoziomowe operacje na plikach oraz transformację danych z formatu dyskowego do struktur pamięciowych.

- **Moduł algorytmów (`layout.c`, `layout.h`):** Implementuje logikę matematyczną odpowiedzialną za wyznaczanie współrzędnych.

2.2 Reprezentacja grafu w pamięci

Wybór struktur danych w pliku `graph.h` został podyktowany koniecznością efektywnego operowania na dużych zbiorach danych przy zachowaniu elastyczności.

2.2.1 Wierzchołki i Krawędzie

Każdy wierzchołek (`Vertex`) posiada unikalny identyfikator `id` oraz współrzędne typu `double` dla zachowania wysokiej precyzji obliczeń. Dodatkowo struktura zawiera pola `force_x` i `force_y`, które służą jako bufory dla wektorów sił w algorytmach iteracyjnych (takich jak Fruchterman-Reingold).

Krawędzie (`Edge`) przechowują informacje o nazwie oraz identyfikatory dwóch łączonych węzłów (`id_A`, `id_B`). Pole `weight` pozwala na uwzględnienie wag w procesie pozycjonowania (np. silniejsze przyciąganie dla krawędzi o większej wadze).

2.2.2 Zarządzanie dynamiczne (Struktura Graph)

Zastosowano model dynamicznych tablic. Struktura `Graph` zawiera wskaźniki na tablice struktur wierzchołków i krawędzi. Pola `capacity_vertices` oraz `capacity_edges` pozwalają na monitorowanie zarezerwowanego miejsca w RAM, co umożliwia automatyczne powiększanie struktur (np. za pomocą `realloc`) podczas wczytywania plików o nieznanej wcześniej liczbie elementów.

2.3 Analiza procesu wykonawczego

Proces działania programu zaimplementowany w `main.c` można podzielić na kilka kluczowych faz:

1. **Analiza argumentów wywołania:** Wykorzystanie biblioteki `unistd.h` i funkcji `getopt` pozwala na obsłużenie parametrów w dowolnej kolejności. Program rygorystycznie sprawdza obecność wymaganych flag (`-i` oraz `-o`).

2. **Ładowanie danych:** Funkcja `load_graph_from_file` otwiera strumień pliku w trybie odczytu. Następuje alokacja pamięci dla głównej struktury grafu. Kod przewiduje sytuację braku pamięci (obsługa błędów alokacji), co jest kluczowe przy pracy na systemach o ograniczonych zasobach.

3. **Wykonanie obliczeń:** Na podstawie wartości zmiennej `algorithm` (wybranej flagą `-a`), program przekazuje wskaźnik do struktury grafu do odpowiedniej funkcji obliczeniowej. Zastosowanie wskaźników do struktur (`Graph *g`) minimalizuje narzut czasowy związany z kopiowaniem danych w pamięci.

4. **Eksport i serializacja:** Funkcja `save_graph_to_file` obsługuje dwa tryby zapisu. W trybie binarnym (`binary_mode`) dane są zrzucane bezpośrednio z pamięci do pliku, co eliminuje błędy zaokrągleń i przyspiesza odczyt w module JAVA.

5. **Zwalnianie zasobów:** Po zakończeniu zapisu wywoływana jest funkcja `free(g)`, która zamyka cykl życia obiektu i zapobiega wyciekom pamięci (*memory leaks*).

2.4 Obsługa błędów i kody wyjścia

Aplikacja implementuje spójny system informowania o błędach poprzez standardowy strumień błędów (`stderr`) oraz kody powrotu do systemu operacyjnego. Dzięki temu program może być łatwo wykorzystywany w skryptach automatyzujących (*batch processing*).

Kod	**Status**	**Sytuacja wyzwalająca**
0	SUCCESS	Program zakończył się pomyślnie.
1	ARG_MISSING	Nie podano pliku wejściowego (`-i`) lub wyjściowego (`-o`).
2	IO_ERROR	Problem z dostępem do pliku (brak pliku lub brak uprawnień).
4	VALID_ERR	Podano nieprawidłową flagę lub numer algorytmu spoza zakresu.
5	MEM_ERR	Krytyczny błąd alokacji pamięci przez funkcję `malloc`.

Table 1: Zestawienie kodów wyjścia aplikacji C.

2.5 Uwagi o implementacji algorytmów

Zgodnie z plikiem `layout.c`, funkcje algorytmiczne zostały przygotowane jako moduły otwarte na rozbudowę. Algorytm **Fruchterman-Reingold** operuje na polach `force_x` i `force_y`, sumując wektory sił dla każdej iteracji przed aktualizacją pozycji `x` i `y`. Algorytm **Tutte'a** wymaga przygotowania macierzy sąsiedztwa i rozwiązania układu równań liniowych, co jest

realizowane poprzez iteracyjne wyznaczanie średniej pozycji wierzchołków sąsiednich względem ustalonej otoczki.